



EXERCISE — Database

version #1.0.0



Copyright

This document is for internal use at EPITA ([website](#)) only.

Copyright © 2025-2026 Assistants [<assistants@tickets.assistants.epita.fr>](mailto:assistants@tickets.assistants.epita.fr)

The use of this document must abide by the following rules:

- ▷ You downloaded it from the assistants' intranet.*
- ▷ This document is strictly personal and must **not** be passed onto someone else.
- ▷ Non-compliance with these rules can lead to severe sanctions.

Contents

1	Goal	5
2	Given annotations	6
2.1	Database annotations	6
2.2	Repository and model management	7
3	How to start	7
3.1	Set-up the database	7
3.2	Create the database	8
3.3	Given Files	8

*<https://intra.forge.epita.fr>

File Tree

```
database/
├─ pom.xml
├─ src/
│   └─ main/
│       ├── java/
│       │   ├── fr/
│       │   │   └─ epita/
│       │   │       ├── assistants/
│       │   │       │   ├── data/
│       │   │       │   │   └─ model/
│       │   │       │   │       ├── CourseModel.java (to submit)
│       │   │       │   │       └─ StudentModel.java (to submit)
│       │   └─ resources/
│       │       └─ application.properties
```

Obligations

Obligations are **fundamental** rules shared by all subjects. They are non-negotiable and to not apply them means to face sanctions. Therefore, do not hesitate to ask for explanations if you do not understand one of these rules.

Obligation #0: Cheating, as well as sharing source code, tests, test tools or coding-style correction tools is **strictly forbidden** and penalized by not being graded, being flagged as a cheater and reported to the academic staff.

Obligation #1: Your submission repository must be **clean**. Except for special cases, which (if any) are **explicitly** mentioned in this document, an *unclean* repository may contain:

- binary files;¹
- files with inappropriate privileges;
- forbidden files: `*~`, `*.swp`, `*.o`, `*.a`, `*.so`, `*.class`, `*.log`, `*.core`, etc.;
- a file tree that does not follow our specifications.

¹If an executable file is required, please provide its sources **only**. We will compile it ourselves.

1 Goal

This exercise will make you explore the `model` layer of Quarkus.

The goal of this exercise is to create models that would generate a simple database when run with the Quarkus framework. You will have to create your models so that they match the database diagram shown below.

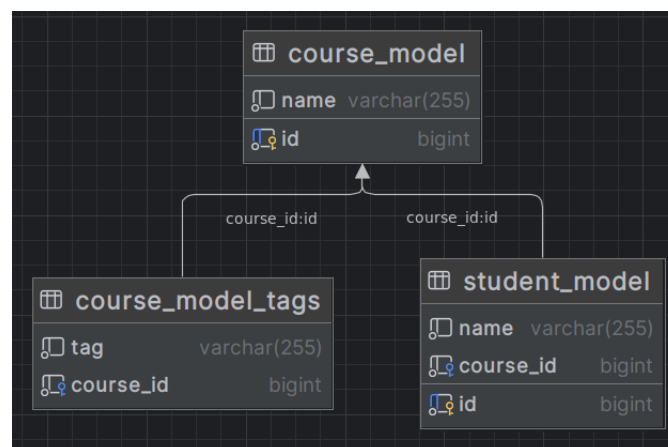
Tips

For projects with Quarkus, databases are never created manually and are instead autogenerated by the **Hibernate ORM** through the use of models.

Be careful!

The only models you have to implement are *course_model* and *student_model*. *course_model_tags* should be created automatically through the `@ElementCollection` annotation.

You do not need to write any database management or other complex behaviors. You do not even need to write any methods for this exercise.



Tips

In order to correctly visualise your database in IntelliJ IDEA, you should open the *Database* tab in IntelliJ IDEA, select your models and select *Diagrams -> Show diagram*

Be careful!

Be careful if you use your own computer, the *Database* tab is only visible if you are using the "Ultimate" version of IntelliJ IDEA (default version on the PIE)

If the diagram view seems incorrect, use the *Diagrams -> Show diagram popup* option instead as the default option tends to not refresh correctly

2 Given annotations

2.1 Database annotations

As said earlier, we will use Hibernate and JPA to handle data persistence for us. You **must** not write SQL queries by yourself in this project.

Here is an example on how to use Hibernate and JPA annotations to create a model.

Tips

We give you some documentation about the annotations used in this example, but you are invited to read this [JPA and Hibernate annotations](#) article.

```
@Entity @Table(name = "student")
class Student() {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Long id;
    public String name;
    public String email;
    public @ElementCollection @CollectionTable(name = "student_roles") List<String> roles;
}
```

To complete this exercise, the following annotations will help you, for the class:

- `@Entity`: Tells Quarkus that this class represents a model
- `@Table`: Lets you specify a name for the table containing a model

To complete this exercise, the following annotations will help you, for the attributes:

- `@Id`: Tells Quarkus that this field is the primary key of your model, you should use `@GeneratedValue` with it
- `@GeneratedValue`: Tells Quarkus to generate this value automatically according to a strategy specified in its arguments, you will need to use the **IDENTITY** strategy. If you want to understand the difference between the different values, you can read [this](#)
- `@Column`: Lets you specify a name for the column containing a field
- `@JoinColumn`: Lets you specify which column Quarkus should use when making a *Join* request
- `@ElementCollection`: Tells Quarkus that it should create a table with multiple elements pointing to this object
- `@CollectionTable`: Lets you specify which table Quarkus should use to put the collection (you may use its *joinColumns* argument to specify which column Quarkus should use to Join the new table)
- `@OneToMany`: Tells Quarkus that this field points to multiple objects from another model (You may use `@JoinColumn` with it)
- `@ManyToOne`: Tells Quarkus that multiple object of this class may point to a single object from another model

Tips

You should use the `@OneToMany` and `@ManyToOne` to define the relationships between `course_model` and `student_model`.

You may find these guides on the `@OneToMany` and the `@ManyToOne` relationships useful.

In the context of this exercise, each course can have several students, and each student has a course.

2.2 Repository and model management

To create your repositories, you should read the following documentation about `PanacheRepositoryBase`: <https://quarkus.io/guides/hibernate-orm-panache>

To update your database, your functions **must** be annotated with `@Transactional`. You will find more information about this annotation in the previous documentation link.

3 How to start

3.1 Set-up the database

First, let us configure some PostgreSQL-specific environment variables:

```
42sh$ echo 'export PGDATA="$HOME/postgres_data"' >> ~/.bashrc
42sh$ echo 'export PGHOST="/tmp"' >> ~/.bashrc
42sh$ echo 'export PGPORT="5432"' >> ~/.bashrc
42sh$ source ~/.bashrc
```

This first line adds a `PGDATA` environment variable to your `.bashrc`, containing the location of your choice to store PostgreSQL's data. The second specifies the host name of the machine on which the server will run. As the value begins with a slash, it will be used as the directory containing a socket on which Postgres will listen. Then, let us initialize a new PostgreSQL database cluster. It will generate default databases and configurations in the `$PGDATA` directory.

```
42sh$ nix-shell -p postgresql
42sh$ initdb --locale "$LANG" -E UTF8
```

Be careful!

If you experience a permission denied error, you may need to restore your rights using `chmod 755 ~`.

3.2 Create the database

You can now start the postgres server by running:

```
42sh$ postgres -k "$PGHOST"
```

By default on the PIE, you do not have a *postgres* superuser. You can create one by running the following command:

```
42sh$ createuser -s postgres
```

The previous command launches the database server, for the following ones, you need to enter a new shell in order to connect to your database.

```
42sh$ psql postgres
```

psql takes the name of the database you want to connect to. Here, we choose the default database, postgres. These commands will allow you to create and own a database named courses:

```
-- Create a database named courses
postgres=# CREATE DATABASE courses OWNER postgres;
-- Connect to the courses database
postgres=# \c courses;
-- Create the courses schema
postgres=# CREATE SCHEMA courses;
-- Exit PostgreSQL
postgres=# \q
```

3.3 Given Files

To begin the project smoothly, you *should* open the given files with IntelliJ before reading the upcoming sections of the subject.

Once your project is opened in your IDE, you can type the following command to start the server:

```
42sh$ mvn quarkus:dev
```

It will automatically create your tables and columns. You can observe them by connecting to your database, in a new shell:

```
42sh$ psql courses
```

and doing these commands:

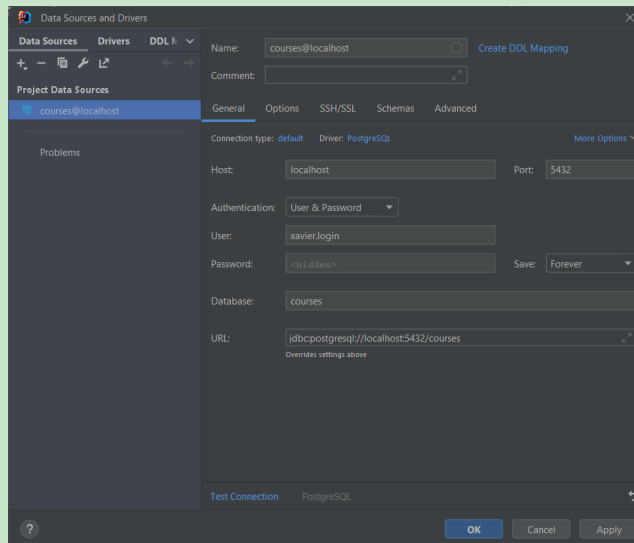
```
-- Show tables
courses=# SELECT tablename FROM pg_tables WHERE schemaname = 'courses';
-- tablename
-- -----
-- course_model
-- course_model_tags
-- student_model
-- (3 rows)
```

(continues on next page)


```
-- Exit PostgreSQL
courses=#$ \q
```

Tips

You can also see the content of your database using IntelliJ IDEA. In order to do so, you must navigate to the *Database* tab, and then click on *new* -> *datasource* -> *postgresql*. And then, you should see the following screen:



Then, you must put *postgres* in the *User* textfield and *courses* in the *Database* one.

Knowledge is a paradox... The more one understands, the more one realizes the vastness of his ignorance.